

SYSTEM PROVISIONING AND MONITORING LAB (CSDV4110P)

LABORATORY MANUAL

Taught by: Dr. Santanu Ghosh

Assistant Professor

CSO, SCOS, UPES

TOTAL MARKS: 50

(10 Experiments · 5 Marks Each)

COURSE OUTCOMES COVERED

C01 · C02 · C03 · C04 · C05



Name _____ Roll No. _____

Programme _____ Semester _____

Session _____ Batch _____

Contents

About This Manual	3
How to Use the Solved Procedures.....	3
Safety, Legal and Cost Notes.....	3
Course Outcomes and Marks Distribution	4
Course Outcomes	4
Experiment to Course Outcome Mapping	4
Standard Structure of Each Experiment.....	4
Experiment I — Infrastructure as Code (IaC) Basics (CO5 · 5 marks).....	6
Experiment II — Advanced IaC Techniques (CO5 · 5 marks).....	15
Experiment III — Continuous Integration for Infrastructure as Code (CO5 · 5 marks)	25
Experiment IV — Designing Effective Monitoring (CO1 and CO2 · 5 marks)	34
Experiment V — Implementing Monitoring Solutions (CO1 and CO2 · 5 marks)	42
Experiment VI — Performance Optimization (CO3 · 5 marks).....	53
Experiment VII — Scalable Monitoring Architectures (CO2 · 5 marks)	62
Experiment VIII — Security Monitoring (CO4 · 5 marks).....	72
Experiment IX — Incident Response Simulation (CO4 · 5 marks).....	82
Experiment X — Emerging Trends and Future Technologies (CO5 · 5 marks).....	93

About This Manual

This manual contains ten laboratory experiments in system provisioning and monitoring. Each experiment is presented in full: the aim and objectives, the hardware and software required, the underlying theory, a detailed step-by-step procedure with the exact commands to be issued, complete program and configuration listings, the output that should be observed at each stage, an observation table, the result, precautions, viva questions with model answers, and the evaluation rubric.

The experiments are cumulative. Experiment III builds a pipeline for the project written in Experiment II; Experiment V implements the plan designed in Experiment IV; Experiment VI measures the system monitored in Experiment V; Experiment VII scales that monitoring; Experiment IX responds to an incident on the detection layer built in Experiment VIII; and Experiment X compares a new technology against everything built in Experiments I to IX. Attempting them out of order will break these dependencies.

Every experiment carries five marks. The total for the laboratory is fifty marks, distributed across five course outcomes as shown in the tables that follow.

How to Use the Solved Procedures

The listings and outputs in this manual are worked reference solutions. They are provided so that a student can recognise a correct result and diagnose a wrong one, not so that they can be copied without execution. Identifiers such as instance IDs, IP addresses, AMI identifiers, timestamps and hashes are illustrative; the values obtained in your own session will differ and it is those values that must be recorded in your lab record.

Where the manual shows a command, run it and record what your system actually printed. Where it shows a measurement, take your own measurement. An experiment record that reproduces this manual's numbers instead of the student's own is treated as not performed.

Safety, Legal and Cost Notes

- Experiments I, II and III create billable cloud resources. Run terraform destroy at the end of every session without exception.
- Never commit credentials, state files or private keys to any repository. Every experiment that uses them explains the correct handling.
- Experiments VIII and IX involve simulated attack traffic. This is permitted only on the isolated laboratory network, only against machines belonging to the laboratory, and only with written approval recorded in advance. Scanning or attempting to authenticate against any other system is an offence under the Information Technology Act, 2000.
- Record the start and end time of every authorised testing window. This record is what distinguishes an exercise from an incident.

Course Outcomes and Marks Distribution

Course Outcomes

CO	Statement	Marks
CO1	Identify critical system metrics, develop monitoring plans, and create dashboards and alerts to ensure the reliable and proactive monitoring of systems.	6
CO2	Design effective monitoring architectures by identifying critical system metrics, developing monitoring plans, and creating dashboards to ensure the reliable and proactive monitoring of systems.	9
CO3	Analyze system performance metrics, identify performance bottlenecks, and implement optimizations to improve system performance.	5
CO4	Apply security monitoring into their overall monitoring strategies.	10
CO5	Demonstrate project development skills.	20
	Total	50

Experiment to Course Outcome Mapping

Exp.	Title	CO	CO1	CO2	CO3	CO4	CO5	Total
I	Infrastructure as Code (IaC) Basics	CO5	–	–	–	–	5	5
II	Advanced IaC Techniques	CO5	–	–	–	–	5	5
III	Continuous Integration for IaC	CO5	–	–	–	–	5	5
IV	Designing Effective Monitoring	CO1, CO2	3	2	–	–	–	5
V	Implementing Monitoring Solutions	CO1, CO2	3	2	–	–	–	5
VI	Performance Optimization	CO3	–	–	5	–	–	5
VII	Scalable Monitoring Architectures	CO2	–	5	–	–	–	5
VIII	Security Monitoring	CO4	–	–	–	5	–	5
IX	Incident Response Simulation	CO4	–	–	–	5	–	5
X	Emerging Trends and Future Technologies	CO5	–	–	–	–	5	5
	TOTAL		6	9	5	10	20	50

Standard Structure of Each Experiment

1. Aim and objectives, with the mapped course outcome and marks.
2. Requirements — hardware, software and accounts needed before starting.
3. Theory — the concepts required to perform and understand the experiment.
4. Procedure — numbered steps with the exact commands to be issued.
5. Program and configuration listings — the complete working code.
6. Expected output — what a correct run produces at each stage.
7. Observation table — to be completed with your own measured values.
8. Result and precautions.
9. Viva questions with model answers.
10. Evaluation rubric showing how the five marks are awarded.

Experiment I — Infrastructure as Code (IaC) Basics

Field	Detail
Course outcome	C05
Marks	5 marks (part of the 20 marks allotted to C05)
Date of performance	_____
Marks awarded	_____

Aim

To introduce the concepts of Infrastructure as Code (IaC), to select an appropriate IaC tool, and to write and execute a simple IaC script that provisions a virtual machine on a cloud platform.

Objectives

- To understand the difference between manual provisioning and provisioning through code.
- To install and configure Terraform and a cloud provider CLI.
- To author a declarative configuration that creates a single virtual machine.
- To execute the complete Terraform lifecycle and interpret the output of each stage.
- To place the configuration under version control and destroy the resource cleanly.

Requirements

Hardware

- Laptop or desktop with a minimum of 8 GB RAM and 20 GB free disk space
- A stable internet connection (provider plugins are downloaded at run time)

Software

- Terraform CLI version 1.6 or later
- AWS CLI version 2 (or Azure CLI if Azure is chosen)
- Git version 2.30 or later
- Visual Studio Code with the HashiCorp Terraform extension
- A terminal — Bash on Linux/macOS or PowerShell on Windows

Accounts and access

- An AWS Free Tier account (or Azure for Students)
- An IAM user with programmatic access and the AmazonEC2FullAccess policy attached
- The access key ID and secret access key for that IAM user

Theory

1.1 What is Infrastructure as Code

Infrastructure as Code is the practice of defining and managing computing infrastructure — virtual machines, networks, storage, load balancers and firewall rules — in machine-readable configuration files rather than through manual configuration in a web console or by running ad-hoc commands on a terminal.

The configuration files are ordinary text files. They are stored in a version control system, reviewed by peers before being merged, tested automatically, and executed by a tool that translates the description into calls against the cloud provider's API. In other words, infrastructure is treated with exactly the same engineering discipline as application source code.

The motivation is repeatability. A virtual machine created by clicking through fourteen screens in a console cannot be recreated identically six months later, and no record exists of who changed what. The same machine described in twenty lines of configuration can be recreated in any account, in any region, by any team member, with a single command.

1.2 Declarative and Imperative Approaches

An imperative approach specifies the sequence of operations required to reach a desired state. A shell script that runs a series of CLI commands is imperative: it says what to do, in what order, and it is the author's responsibility to handle the case where a resource already exists.

A declarative approach specifies the desired end state and leaves the tool to determine the operations needed to reach it. Terraform, AWS CloudFormation and Kubernetes manifests are declarative. The author writes "there should be one virtual machine of this size with these tags"; the tool inspects what currently exists, computes the difference, and performs only the operations required to close that difference.

The declarative model is preferred for infrastructure because the same configuration file is valid whether it is being applied to an empty account or to an account where the infrastructure already exists.

1.3 Architecture of Terraform

Terraform consists of a core binary and a set of provider plugins. The core parses the configuration, builds a dependency graph, computes the execution plan and manages state. Providers translate the generic resource operations into API calls for a particular platform — the AWS provider, the Azure provider, the Kubernetes provider and several hundred others.

Providers are not shipped with the binary. The terraform init command reads the required_providers block, downloads the matching plugin from the Terraform Registry into a local .terraform directory, and records the exact version and checksum in a lock file so that subsequent runs are reproducible.

1.4 State and Configuration Drift

Terraform records the mapping between the resources declared in the configuration and the real objects created in the cloud in a state file, by default terraform.tfstate in the working directory. The state file stores the cloud-assigned identifiers, such as the EC2 instance ID, that cannot be derived from the configuration alone.

Because the state file may contain sensitive values, it must never be committed to a public repository. In team environments the state is stored remotely — in an S3 bucket with DynamoDB locking, or in Terraform Cloud — so that concurrent runs cannot corrupt it.

Configuration drift is any difference between the state recorded by Terraform and the actual configuration of the resource in the cloud, usually caused by someone making a manual change in the console. Running terraform plan refreshes the state against the live API and reports drift as a proposed change that would restore the declared configuration.

1.5 Idempotency

An operation is idempotent if performing it repeatedly produces the same result as performing it once. Terraform apply is idempotent: applying an unchanged configuration a second time reports "No changes. Your infrastructure matches the configuration" and creates nothing.

Idempotency is what makes automated infrastructure safe. A continuous integration pipeline can apply the configuration on every commit without any risk of accumulating duplicate resources.

1.6 The Terraform Workflow

The standard workflow consists of five commands. `terraform init` prepares the working directory and downloads providers. `terraform fmt` rewrites the configuration into the canonical style. `terraform validate` checks syntax and internal consistency without contacting the cloud. `terraform plan` refreshes state, computes the difference and prints the proposed actions. `terraform apply` executes the plan after confirmation. `terraform destroy` removes every resource recorded in the state.

Procedure

Step 1 — Install Terraform and verify the installation

Download the Terraform binary for your operating system, extract it and place it on the system PATH. Confirm the installation before proceeding; an incorrect PATH is the most common cause of failure in this experiment.

Verifying the Terraform installation

```
$ terraform -version
Terraform v1.9.5
on linux_amd64
```

Step 2 — Install and configure the AWS CLI

Install AWS CLI version 2 and configure it with the access key of the IAM user created for this laboratory. The credentials are written to `~/.aws/credentials` and are picked up automatically by the Terraform AWS provider.

Configuring and verifying cloud credentials

```
$ aws configure
AWS Access Key ID [None]: AKIA*****
AWS Secret Access Key [None]: *****
Default region name [None]: ap-south-1
Default output format [None]: json

$ aws sts get-caller-identity
{
  "UserId": "AIDA*****",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/spm-lab-user"
}
```

Step 3 — Create the project directory and initialise version control

Create a dedicated directory for the experiment, initialise a Git repository inside it and add a `.gitignore` file before writing any configuration. Adding `.gitignore` first guarantees that a state file can never be committed by accident.

Creating the workspace

```
$ mkdir exp1-iac-basics && cd exp1-iac-basics
$ git init
Initialized empty Git repository in /home/student/exp1-iac-basics/.git/
$ printf '*.tfstate\n*.tfstate.backup\n*.terraform\n*.tfvars\n' > .gitignore
```

Step 4 — Write the provider configuration

Create `main.tf` and declare the terraform block with the required provider and its version constraint, followed by the provider block. The constraint `"~> 5.0"` permits any 5.x version but not 6.0, which prevents a breaking provider release from silently changing the behaviour of the configuration.

Step 5 — Declare input variables

Create `variables.tf` and declare the region, instance type and machine name as input variables with sensible defaults. Parameterising these values is what makes the configuration reusable across regions and across students.

Step 6 — Declare the virtual machine resource

Add the `aws_instance` resource to `main.tf`. The resource block has two labels: the resource type, which is fixed by the provider, and a local name used to reference the resource elsewhere in the configuration. Tags are not optional in practice — untagged resources cannot be attributed to an owner and are the main cause of runaway cloud bills in student accounts.

Step 7 — Declare outputs

Create `outputs.tf` and export the instance identifier and the public IP address. Outputs are printed at the end of a successful apply and can be queried later with `terraform output`, which avoids having to open the cloud console to find the address.

Step 8 — Initialise the working directory

Run `terraform init`. The command downloads the AWS provider plugin, creates the `.terraform` directory and writes `.terraform.lock.hcl`. Commit the lock file to Git; it pins the provider version and its checksum for every future run.

Output of `terraform init`

```
$ terraform init

Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.62.0...
- Installed hashicorp/aws v5.62.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above.

Terraform has been successfully initialized!
```

Step 9 — Format and validate the configuration

Run `terraform fmt` to normalise indentation and alignment, then `terraform validate` to check that the configuration is syntactically correct and internally consistent. Validation does not contact AWS, so it runs in under a second and should be used continuously while editing.

Formatting and validating

```
$ terraform fmt
main.tf
variables.tf

$ terraform validate
Success! The configuration is valid.
```

Step 10 — Generate and review the execution plan

Run `terraform plan`. Read every line of the output. The symbol `+` indicates creation, `~` indicates in-place modification, and `-/+` indicates that the resource must be destroyed and recreated. Values shown as (known after apply) are assigned by AWS and cannot be predicted. The summary line at the end must read exactly "Plan: 1 to add, 0 to change, 0 to destroy".

Step 11 — Apply the configuration

Run `terraform apply`. Terraform recomputes the plan, displays it and waits for confirmation. Type `yes` at the prompt. Creation of an EC2 instance normally completes in thirty to sixty seconds. Record the instance ID and public IP printed in the outputs section.

Step 12 — Verify the running instance

Confirm the result independently of Terraform, both through the AWS CLI and through the EC2 section of the AWS Management Console. Verification through a second channel is essential; a successful apply proves that the API call succeeded, not that the machine is in the expected state.

Independent verification through the AWS CLI

```
$ terraform output
instance_id = "i-0a1b2c3d4e5f67890"
public_ip   = "13.234.56.78"

$ aws ec2 describe-instances \
  --instance-ids i-0a1b2c3d4e5f67890 \
  --query 'Reservations[0].Instances[0].[InstanceId,State.Name,InstanceType]' \
  --output text
i-0a1b2c3d4e5f67890    running    t2.micro
```

Step 13 — Commit the configuration

Stage and commit the .tf files and the lock file. Verify with git status that no state file or credential file has been staged.

Committing the configuration

```
$ git add main.tf variables.tf outputs.tf .gitignore .terraform.lock.hcl
$ git commit -m 'Experiment I: provision a single EC2 instance with Terraform'
$ git status --short
(no output – working tree clean, state file correctly ignored)
```

Step 14 — Destroy the infrastructure

Run terraform destroy and confirm with yes. This step is mandatory. A t2.micro instance left running consumes the Free Tier allowance and will be billed once the allowance is exhausted. Verify in the console that the instance state has changed to terminated.

Program and Configuration Listings

Listing 1.1 — main.tf

```
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.region
}

# A single Ubuntu virtual machine of free-tier size.
resource "aws_instance" "lab_vm" {
  ami           = var.ami_id
  instance_type = var.instance_type

  tags = {
    Name        = var.vm_name
    Owner       = "SPM-Lab"
    Experiment   = "01"
    ManagedBy    = "Terraform"
  }
}
```

Listing 1.2 — variables.tf

```

variable "region" {
  description = "AWS region in which the instance is created"
  type       = string
  default    = "ap-south-1"
}

variable "ami_id" {
  description = "AMI identifier of Ubuntu 22.04 LTS in the chosen region"
  type       = string
  default    = "ami-0f58b397bc5c1f2e8"
}

variable "instance_type" {
  description = "EC2 instance size; must remain within the free tier"
  type       = string
  default    = "t2.micro"

  validation {
    condition = contains(["t2.micro", "t3.micro"], var.instance_type)
    error_message = "Only free-tier instance types are permitted in this lab."
  }
}

variable "vm_name" {
  description = "Value of the Name tag applied to the instance"
  type       = string
  default    = "lab-vm-01"
}

```

Listing 1.3 — outputs.tf

```

output "instance_id" {
  description = "Identifier assigned to the instance by AWS"
  value      = aws_instance.lab_vm.id
}

output "public_ip" {
  description = "Public IPv4 address of the instance"
  value      = aws_instance.lab_vm.public_ip
}

output "availability_zone" {
  description = "Availability zone chosen by AWS"
  value      = aws_instance.lab_vm.availability_zone
}

```

Expected Output

Output 1.1 — terraform plan (abridged)

```

$ terraform plan

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.lab_vm will be created
+ resource "aws_instance" "lab_vm" {
  + ami              = "ami-0f58b397bc5c1f2e8"
  + arn              = (known after apply)
  + availability_zone = (known after apply)
  + id               = (known after apply)
  + instance_type    = "t2.micro"
  + private_ip       = (known after apply)
  + public_ip        = (known after apply)
  + tags              = {
    + "Experiment" = "01"
    + "ManagedBy" = "Terraform"
    + "Name"       = "lab-vm-01"
    + "Owner"      = "SPM-Lab"
  }
}

Plan: 1 to add, 0 to change, 0 to destroy.

```

Output 1.2 — terraform apply and terraform destroy

```

$ terraform apply
... (plan repeated) ...
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_instance.lab_vm: Creating...
aws_instance.lab_vm: Still creating... [10s elapsed]
aws_instance.lab_vm: Still creating... [20s elapsed]
aws_instance.lab_vm: Creation complete after 32s [id=i-0a1b2c3d4e5f67890]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:
availability_zone = "ap-south-1a"
instance_id      = "i-0a1b2c3d4e5f67890"
public_ip       = "13.234.56.78"

$ terraform destroy
Enter a value: yes
aws_instance.lab_vm: Destroying... [id=i-0a1b2c3d4e5f67890]
aws_instance.lab_vm: Destruction complete after 41s

Destroy complete! Resources: 1 destroyed.

```

Observation Table

Stage	Command	Observed result
Initialisation	terraform init	AWS provider v5.62.0 installed; lock file created
Validation	terraform validate	Success! The configuration is valid.
Planning	terraform plan	Plan: 1 to add, 0 to change, 0 to destroy
Provisioning	terraform apply	1 resource added in 32 seconds
Verification	aws ec2 describe-instances	State reported as running, type t2.micro
Idempotency check	terraform apply (second run)	No changes. Infrastructure matches the configuration
Teardown	terraform destroy	1 resource destroyed; state file emptied

Result

A single Linux virtual machine was successfully provisioned on AWS from a declarative Terraform configuration, verified through both the CLI and the management console, and destroyed cleanly. The concepts of declarative infrastructure, idempotency and state management were demonstrated in practice.

Precautions

- Never commit terraform.tfstate, terraform.tfvars or any credential file to a Git repository. Verify with git status before every commit.
- Always read the full output of terraform plan before typing yes. The plan is the only safety gate between the configuration and the cloud account.
- Use only free-tier instance types. The validation block in variables.tf enforces this, but the check can be bypassed by editing the file.

- Run terraform destroy at the end of every session. An instance left running overnight consumes free-tier hours that are shared across the whole month.
- AMI identifiers are specific to a region and are periodically rotated. If apply fails with an InvalidAMIID.NotFound error, obtain a current identifier before retrying.

Viva Questions with Answers

Q1. What is the difference between terraform plan and terraform apply?

Ans.

Q2. What is stored in the state file and why is it sensitive?

Ans.

Q3. Define idempotency and give an example from this experiment.

Ans.

Q4. What is configuration drift and how does Terraform detect it?

Ans.

Q5. Why is a version constraint placed on the provider?

Ans.

Q6. What is the purpose of the .terraform.lock.hcl file?

Ans.

Q7. Why is the declarative model preferred over an imperative shell script?

Ans.

Evaluation Rubric

#	Evaluation criterion	Marks	CO
1	Tool installation, cloud authentication and workspace setup	1	CO5
2	Correctness and parameterisation of the IaC script	1.5	CO5
3	Successful apply with a verified running VM and a clean destroy	1.5	CO5
4	Lab record, version-control hygiene and viva	1	CO5
	TOTAL	5	CO5

Signature of the faculty in-charge: _____